Applications for Mobile Devices

Data and File Storage

Dr. Vítor Luiz da Silva Verbel
vitor.dasilva@udl.cat

Course: 105025-2526

Universitat de Lleida

Campus Universitari Igualada - UdL

Contents

- Overview
- Media Store
- Coil
- Hands on – External Files
- Hands on – Trip Gallery

Before we go

Before we go

- Download at Virtual Campus: /source examples/TripGallery.zip
- Download at Virtual Campus: /source examples/ReadExternalMediaFilesAPI35.zip

Before we go - Enquesta

https://esecretaria.udl.cat/ServiciosApp

Overview

Data and file storage overview

Android uses a file system that's similar to disk-based file systems on other platforms. The system provides several options for you to save your app data:

- **App-specific storage:** Store files that are meant for your app's use only, either in dedicated directories within an internal storage volume or different dedicated directories within external storage. Use the directories within internal storage to save sensitive information that other apps shouldn't access.
- **Shared storage:** Store files that your app intends to share with other apps, including media, documents, and other files.
- **Preferences:** Store private, primitive data in key-value pairs.
- **Databases:** Store structured data in a private database using the Room persistence library.

https://developer.android.com/training/data-storage

Data and file storage overview

	Type of content	Access method	Permissions needed	Can other apps access?	Files removed on app uninstall?
App-specific files	Files meant for your app's use only	From internal storage, `getFilesDir()` or `getCacheDir()` From external storage, `getExternalFilesDir()` or `getExternalCacheDir()`	Never needed for internal storage Not needed for external storage when your app is used on devices that run Android 4.4 (API level 19) or higher	No	Yes
Media	Shareable media files (images, audio files, videos)	`MediaStore` API	`READ_EXTERNAL_STORAGE` when accessing other apps' files on Android 11 (API level 30) or higher `READ_EXTERNAL_STORAGE` or `WRITE_EXTERNAL_STORAGE` when accessing other apps' files on Android 10 (API level 29) Permissions are required for all files on Android 9 (API level 28) or lower	Yes, though the other app needs the `READ_EXTERNAL_STORAGE` permission	No
Documents and other files	Other types of shareable content, including downloaded files	Storage Access Framework	None	Yes, through the system file picker	No
App preferences	Key-value pairs	Jetpack Preferences library	None	No	Yes
Database	Structured data	Room persistence library	None	No	Yes

The solution you choose depends on your specific needs:

How much space does your data require?

Internal storage has limited space for app-specific data. Use other types of storage if you need to save a substantial amount of data.

How reliable does data access need to be?

If your app's basic functionality requires certain data, such as when your app is starting up, place the data within internal storage directory or a database. App-specific files that are stored in external storage aren't always accessible because some devices allow users to remove a physical device that corresponds to external storage.

What kind of data do you need to store?

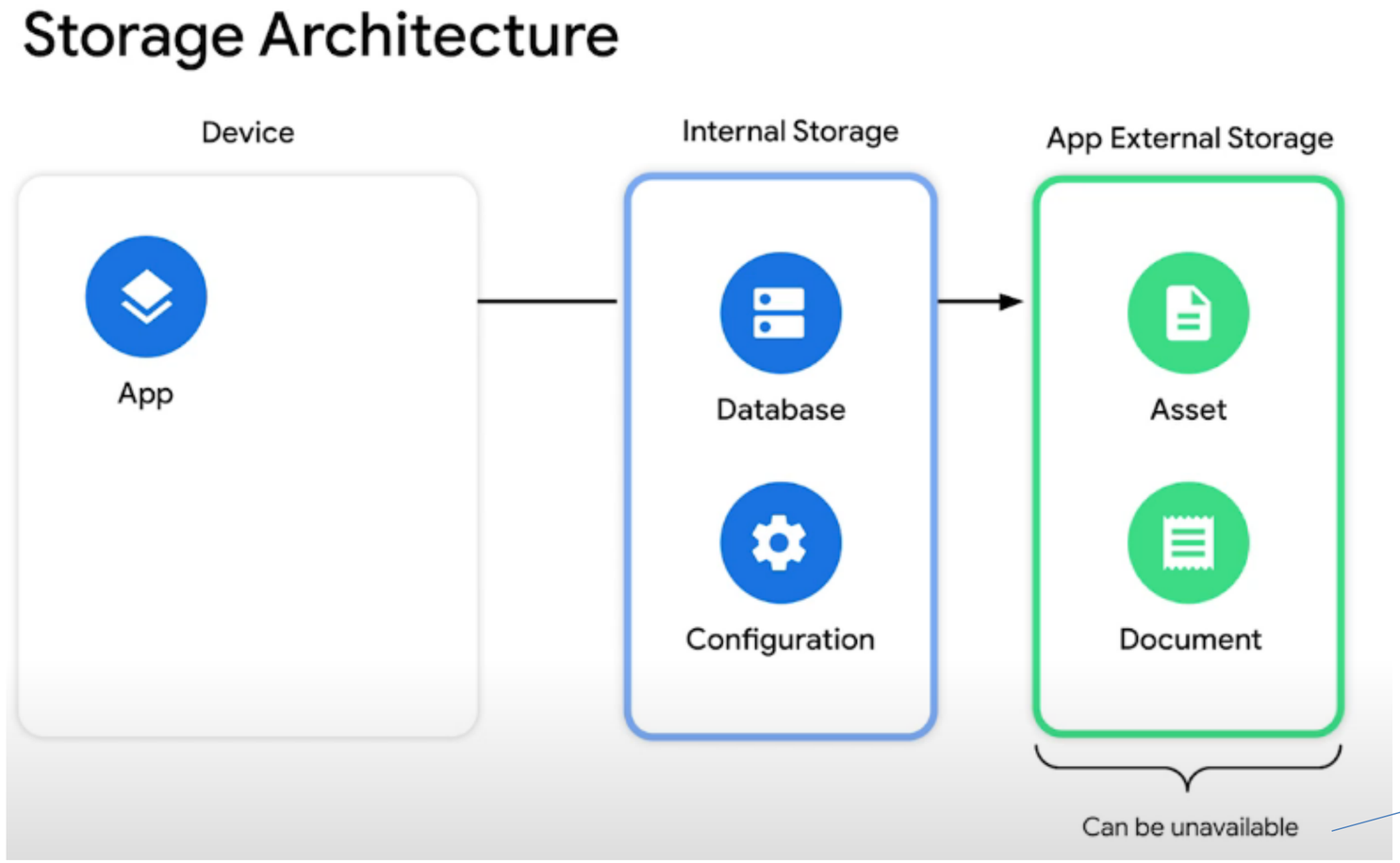
If you have data that's only meaningful for your app, use app-specific storage. For shareable media content, use shared storage so that other apps can access the content. For structured data, use either preferences (for key-value data) or a database (for data that contains more than 2 columns).

Should the data be private to your app?

When storing sensitive data—data that shouldn't be accessible from any other app—use internal storage, preferences, or a database. Internal storage has the added benefit of the data being hidden from users.

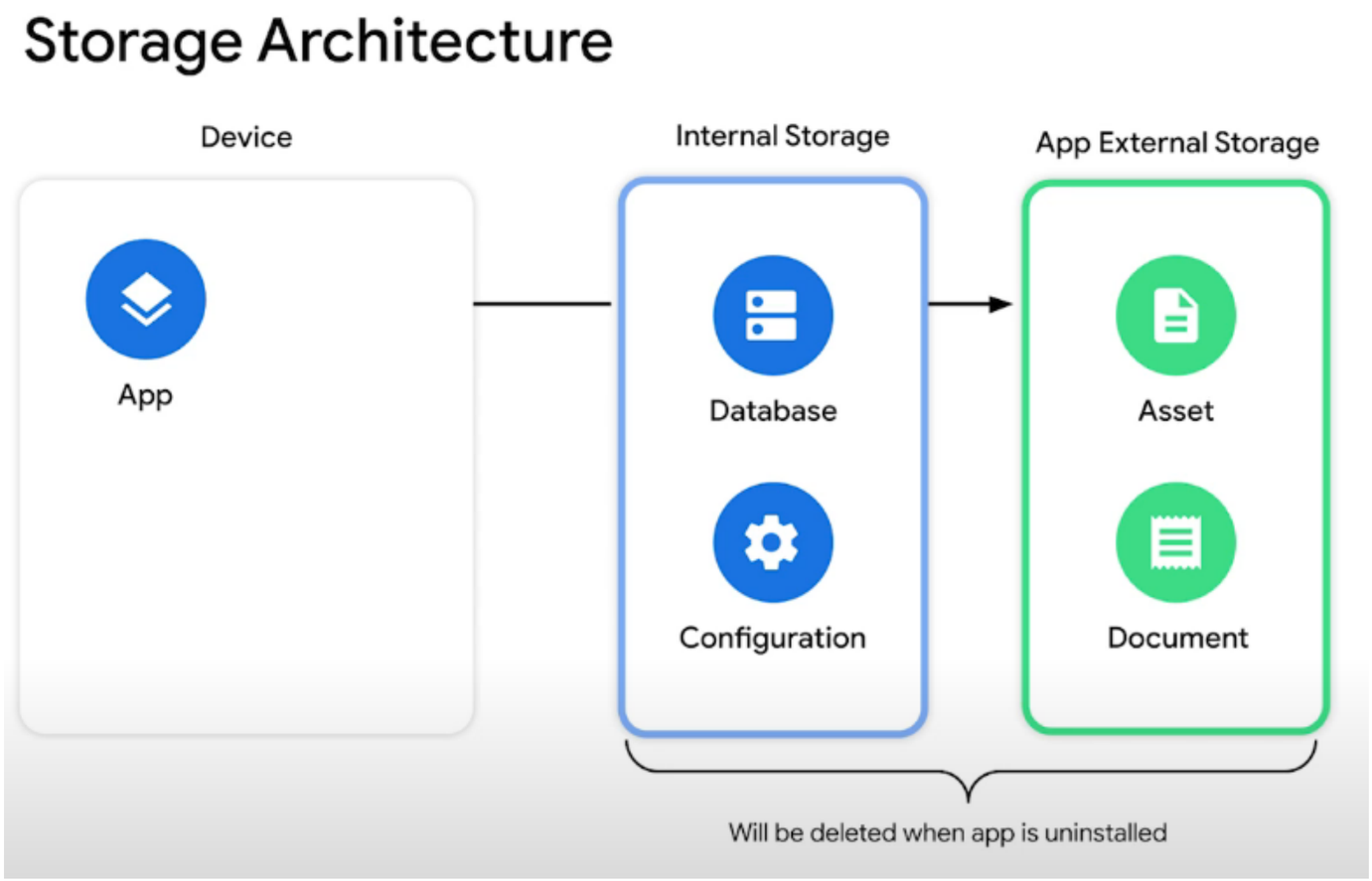
https://developer.android.com/training/data-storage

Storage Architecture



Yacine Rezgui, Abhijeet Kaur
https://www.youtube.com/watch?v=jcO6p5TlcGs

Storage Architecture



Yacine Rezgui, Abhijeet Kaur
https://www.youtube.com/watch?v=jcO6p5TlcGs

Access app-specific files

In many cases, your app creates files that other apps don't need to access, or shouldn't access. The system provides the following locations for storing such *app-specific* files:

- **Internal storage directories:** These directories include both a dedicated location for storing persistent files, and another location for storing cache data. The system prevents other apps from accessing these locations, and on Android 10 (API level 29) and higher, these locations are encrypted. These characteristics make these locations a good place to store sensitive data that only your app itself can access.
- **External storage directories:** These directories include both a dedicated location for storing persistent files, and another location for storing cache data. Although it's possible for another app to access these directories if that app has the proper permissions, the files stored in these directories are meant for use only by your app. If you specifically intend to create files that other apps should be able to access, your app should store these files in the shared storage part of external storage instead.

When the user uninstalls your app, the files saved in app-specific storage are removed. Because of this behavior, you shouldn't use this storage to save anything that the user expects to persist independently of your app. For example, if your app allows users to capture photos, the user would expect that they can access those photos even after they uninstall your app. So you should instead use shared storage to save those types of files to the appropriate media collection.

Access and store files

You can use the `File` API to access and store files.

To help maintain your app's performance, don't open and close the same file multiple times.

The following code snippet demonstrates how to use the `File` API:

Kotlin Java

```kotlin
val file = File(context.filesDir, filename)
```

Store a file using a stream

As an alternative to using the `File` API, you can call `openFileOutput()` to get a `FileOutputStream` that writes to a file within the `filesDir` directory.

The following code snippet shows how to write some text to a file:

Kotlin Java

```kotlin
val filename = "myfile"
val fileContents = "Hello world!"
context.openFileOutput(filename, Context.MODE_PRIVATE).use {
        it.write(fileContents.toByteArray())
}
```

https://developer.android.com/training/data-storage/app-specific

Access app-specific files (external storage)

Verify that storage is available

Because external storage resides on a physical volume that the user might be able to remove, verify that the volume is accessible before trying to read app-specific data from, or write app-specific data to, external storage.

You can query the volume's state by calling `Environment.getExternalStorageState()`. If the returned state is `MEDIA_MOUNTED`, then you can read and write app-specific files within external storage. If it's `MEDIA_MOUNTED_READ_ONLY`, you can only read these files.

For example, the following methods are useful to determine the storage availability:

Kotlin Java

```kotlin
// Checks if a volume containing external storage is available
// for read and write.
fun isExternalStorageWritable(): Boolean {
    return Environment.getExternalStorageState() == Environment.MEDIA_MOUNTED
}

// Checks if a volume containing external storage is available to at least read.
fun isExternalStorageReadable(): Boolean {
     return Environment.getExternalStorageState() in
        setOf(Environment.MEDIA_MOUNTED, Environment.MEDIA_MOUNTED_READ_ONLY)
}
```

Access persistent files

To access app-specific files from external storage, call `getExternalFilesDir()`.

To help maintain your app's performance, don't open and close the same file multiple times.

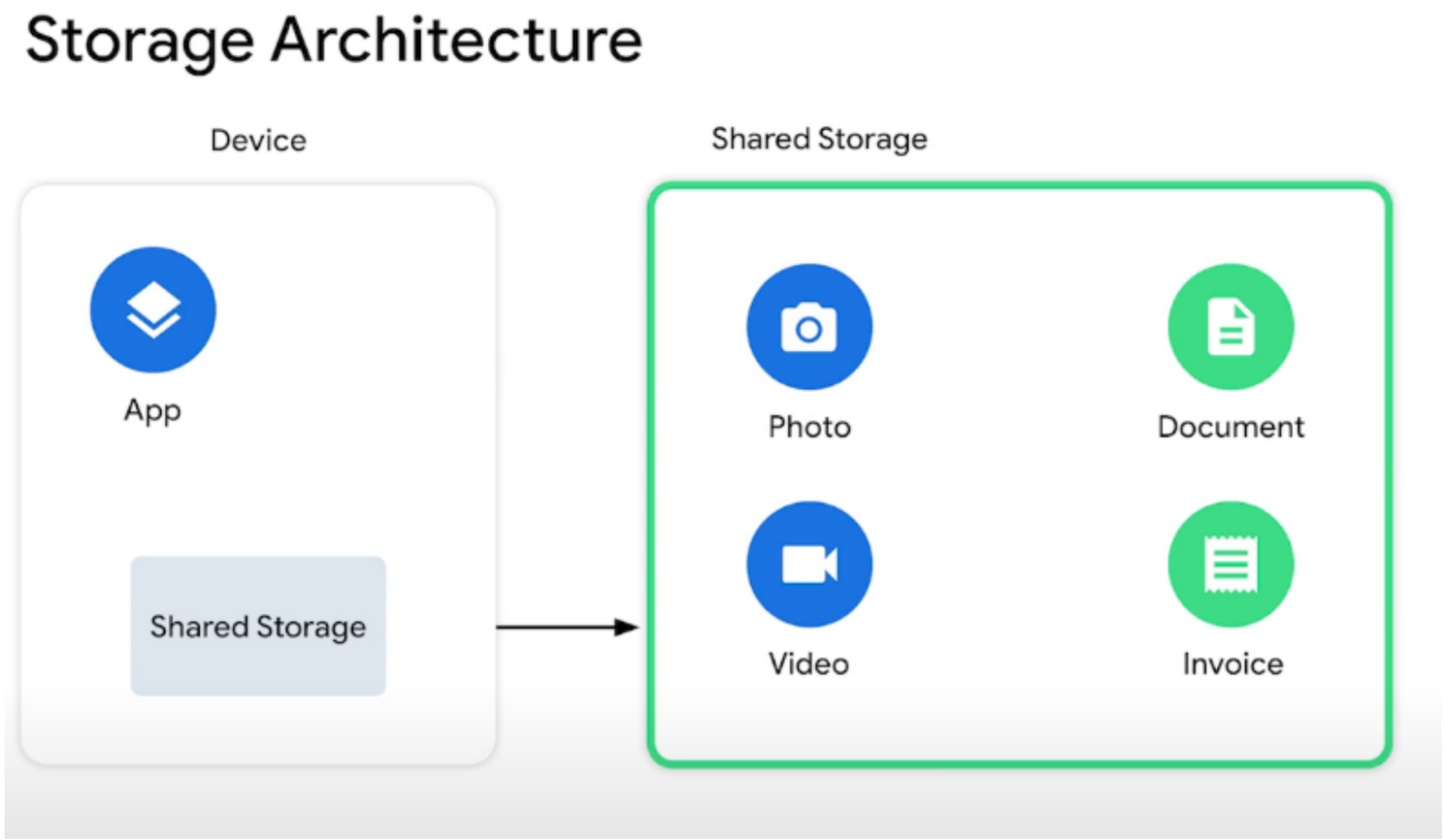
The following code snippet demonstrates how to call `getExternalFilesDir()`:

Kotlin Java

```kotlin
val appSpecificExternalDir = File(context.getExternalFilesDir(null), filename)
```

https://developer.android.com/training/data-storage/app-specific

Shared Storage



Yacine Rezgui, Abhijeet Kaur
https://www.youtube.com/watch?v=jcO6p5TlcGs

Permissions

Permissions and access to external storage

Android defines the following storage-related permissions: `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`, and `MANAGE_EXTERNAL_STORAGE`.

On earlier versions of Android, apps needed to declare the `READ_EXTERNAL_STORAGE` permission to access any file outside the app-specific directories on external storage. Also, apps needed to declare the `WRITE_EXTERNAL_STORAGE` permission to write to any file outside the app-specific directory.

More recent versions of Android rely more on a file's purpose than its location for determining an app's ability to access, and write to, a given file. In particular, if your app targets Android 11 (API level 30) or higher, the `WRITE_EXTERNAL_STORAGE` permission doesn't have any effect on your app's access to storage. This purpose-based storage model improves user privacy because apps are given access only to the areas of the device's file system that they actually use.

Android 11 introduces the `MANAGE_EXTERNAL_STORAGE` permission, which provides write access to files outside the app-specific directory and `MediaStore`. To learn more about this permission, and why most apps don't need to declare it to fulfill their use cases, see the guide on how to manage all files on a storage device.

yRepository.kt | HomeScreen.kt | HomeScreen2.kt | MainActivity.kt | AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

<!--    <uses-permission android:name="android.permission.CAMERA"/>-->
<!--    <uses-feature android:name="android.hardware.camera" android:required="false" />-->

    <uses-permission android:name="android.permission.INTERNET" />



    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
    <uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission android:name="android.permission.READ_MEDIA_VISUAL_USER_SELECTED" />


    <uses-feature android:name="android.hardware.camera.any" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"  android:maxSdkVersion="28" />
```

https://developer.android.com/training/data-storage#permissions

Shared Storage

Use shared storage for user data that can or should be accessible to other apps and saved even if the user uninstalls your app.

Android provides APIs for storing and accessing the following types of shareable data:

- **Media content:** The system provides standard public directories for these kinds of files, so the user has a common location for all their photos, another common location for all their music and audio files, and so on. Your app can access this content using the platform's `MediaStore` API.
- **Documents and other files:** The system has a special directory for containing other file types, such as PDF documents and books that use the EPUB format. Your app can access these files using the platform's Storage Access Framework.
- **Datasets:** On Android 11 (API level 30) and higher, the system caches large datasets that multiple apps might use. These datasets can support use cases like machine learning and media playback. Apps can access these shared datasets using the `BlobStoreManager` API.

Yacine Rezgui, Abhijeet Kaur
https://www.youtube.com/watch?v=jcO6p5TlcGs

Media Store

Photo picker

As an alternative to using the media store, the Android photo picker tool provides a safe, built-in way for users to select media files without needing to grant your app access to their entire media library. This is only available on supported devices. For more information, see the photo picker guide.

Media store

To interact with the media store abstraction, use a `ContentResolver` object that you retrieve from your app's context:

Kotlin Java

```kotlin
val projection = arrayOf(media-database-columns-to-retrieve)
val selection = sql-where-clause-with-placeholder-variables
val selectionArgs = values-of-placeholder-variables
val sortOrder = sql-order-by-clause

applicationContext.contentResolver.query(
    MediaStore.media-type.Media.EXTERNAL_CONTENT_URI,
    projection,
    selection,
    selectionArgs,
    sortOrder
)?.use { cursor ->
    while (cursor.moveToNext()) {
        // Use an ID column from the projection to get
        // a URI representing the media item itself.
    }
}
```

Copy code sample

The system automatically scans an external storage volume and adds media files to the following well-defined collections:

- **Images,** including photographs and screenshots, which are stored in the `DCIM/` and `Pictures/` directories. The system adds these files to the `MediaStore.Images` table.
- **Videos,** which are stored in the `DCIM/`, `Movies/`, and `Pictures/` directories. The system adds these files to the `MediaStore.Video` table.
- **Audio files,** which are stored in the `Alarms/`, `Audiobooks/`, `Music/`, `Notifications/`, `Podcasts/`, and `Ringtones/` directories. Additionally, the system recognizes audio playlists that are in the `Music/` or `Movies/` directories as well as voice recordings that are in the `Recordings/` directory. The system adds these files to the `MediaStore.Audio` table. *The `Recordings/` directory isn't available on Android 11 (API level 30) and lower.*
- **Downloaded files,** which are stored in the `Download/` directory. On devices that run Android 10 (API level 29) and higher, these files are stored in the `MediaStore.Downloads` table. *This table isn't available on Android 9 (API level 28) and lower.*

The media store also includes a collection called `MediaStore.Files`. Its contents depend on whether your app uses scoped storage, available on apps that target Android 10 or higher.

- If scoped storage is enabled, the collection shows only the photos, videos, and audio files that your app has created. Most developers don't need to use `MediaStore.Files` to view media files from other apps, but if you have a specific requirement to do so, you can declare the `READ_EXTERNAL_STORAGE` permission. We recommend, however, that you use the `MediaStore` APIs to open files that your app hasn't created.
- If scoped storage is unavailable or not being used, the collection shows all types of media files.

https://developer.android.com/training/data-storage/shared/media

Media Store - Permissions

Request necessary permissions

Before performing operations on media files, make sure your app has declared the permissions that it needs to access these files. Be careful, however, not to declare permissions that your app doesn't need or use.

Storage permissions

Whether your app needs permissions to access storage depends on whether it accesses only its own media files or files created by other apps.

Access your own media files

On devices that run Android 10 or higher, you don't need storage-related permissions to access and modify media files that your app owns, including files in the `MediaStore.Downloads` collection. If you're developing a camera app, for example, you don't need to request storage-related permissions to access the photos it takes, because your app owns the images that you're writing to the media store.

Access other apps' media files

To access media files that other apps create, you must declare the appropriate storage-related permissions, and the files must reside in one of the following media collections:

- `MediaStore.Images`
- `MediaStore.Video`
- `MediaStore.Audio`

As long as a file is viewable from the `MediaStore.Images`, `MediaStore.Video`, or `MediaStore.Audio` queries, it's also viewable using the `MediaStore.Files` query.

Access other apps' media files

To access media files that other apps create, you must declare the appropriate storage-related permissions, and the files must reside in one of the following media collections:

- `MediaStore.Images`
- `MediaStore.Video`
- `MediaStore.Audio`

As long as a file is viewable from the `MediaStore.Images`, `MediaStore.Video`, or `MediaStore.Audio` queries, it's also viewable using the `MediaStore.Files` query.

The following code snippet demonstrates how to declare the appropriate storage permissions:

```
<!-- Required only if your app needs to access images or photos
     that other apps created. -->
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />

<!-- Required only if your app needs to access videos
     that other apps created. -->
<uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />

<!-- Required only if your app needs to access audio files
     that other apps created. -->
<uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />

<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>

<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
                 android:maxSdkVersion="29" />
```

https://developer.android.com/training/data-storage/shared/media

Media Store – Permissions (Legacy/Older versions)

Extra permissions needed for apps running on legacy devices

If your app is used on a device that runs Android 9 or lower, or if your app has temporarily opted out of scoped storage, you must request the `READ_EXTERNAL_STORAGE` permission to access any media file. If you want to modify media files, you must request the `WRITE_EXTERNAL_STORAGE` permission, as well.

Storage Access Framework required for accessing other apps' downloads

If your app wants to access a file within the `MediaStore.Downloads` collection that your app didn't create, you must use the Storage Access Framework. To learn more about how to use this framework, see Access documents and other files from shared storage.

Media location permission

If your app targets Android 10 (API level 29) or higher and needs to retrieve unredacted EXIF metadata from photos, you need to declare the `ACCESS_MEDIA_LOCATION` permission in your app's manifest, then request this permission at runtime.

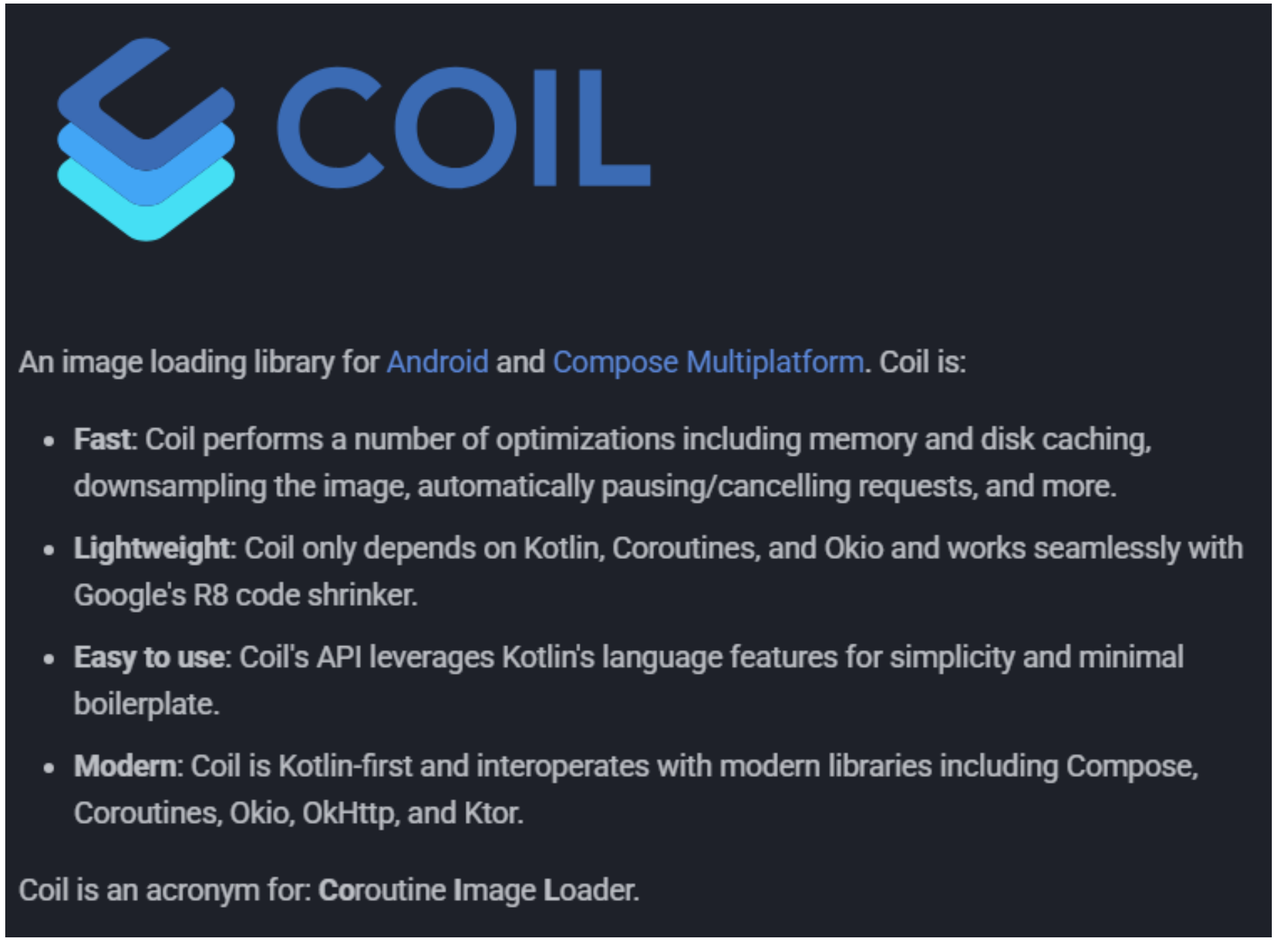
yRepository.kt | HomeScreen.kt | HomeScreen2.kt | MainActivity.kt | AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

<!--    <uses-permission android:name="android.permission.CAMERA"/>-->
<!--    <uses-feature android:name="android.hardware.camera" android:required="false" />-->

    <uses-permission android:name="android.permission.INTERNET" />



    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
    <uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission android:name="android.permission.READ_MEDIA_VISUAL_USER_SELECTED" />


    <uses-feature android:name="android.hardware.camera.any" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"  android:maxSdkVersion="28" />
```

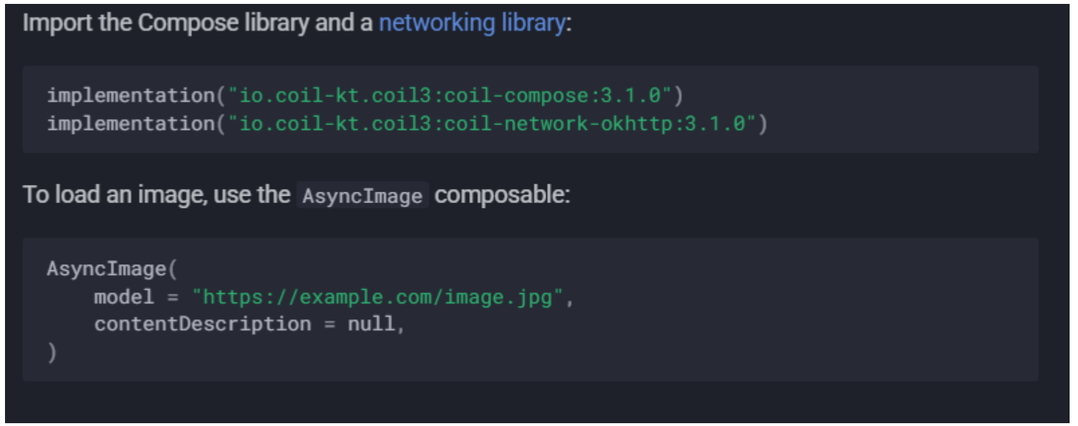
https://developer.android.com/training/data-storage/shared/media

Coil

COIL – **Co**routine **I**mage **L**oader



https://coil-kt.github.io/coil/



Hands On – Read External Files

Code Review

- Download /code/ReadExternalMediaFilesAPI35.zip

Code Review – MainActivity (Permissions)

```kotlin
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()

    val permissions = if(Build.VERSION.SDK_INT >= 33) {
        arrayOf(
            Manifest.permission.READ_MEDIA_AUDIO,
            Manifest.permission.READ_MEDIA_VIDEO,
            Manifest.permission.READ_MEDIA_IMAGES,
        )
    } else {
        arrayOf(Manifest.permission.READ_EXTERNAL_STORAGE)
    }

    ActivityCompat.requestPermissions(
        activity: this,
        permissions,
        requestCode: 0
    )

    setContent {
        ReadExternalMediaFilesAPI35Theme {
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                LazyColumn(
                    modifier = Modifier
                        .fillMaxSize()
                        .padding(innerPadding)
                ) {
                    items(viewModel.files) {
                        MediaListItem(
                            file = it,
                            modifier = Modifier
                                .fillMaxWidth()
                        )
                    }
                }
            }
        }
    }
}
```

MainActivity.kt | AndroidManifest.xml | MediaFile.kt

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.READ_MEDIA_AUDIO" />
    <uses-permission android:name="android.permission.READ_MEDIA_VIDEO" />
    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
    <uses-permission android:name="android.permission.READ_MEDIA_VISUAL_USER_SELECTED" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="ReadExternalMediaFilesAPI35"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ReadExternalMediaFilesAPI35"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="ReadExternalMediaFilesAPI35"
            android:theme="@style/Theme.ReadExternalMediaFilesAPI35">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

List Files

MainActivity.kt | AndroidManifest.xml | MediaListItem.kt | MediaFile.kt

```kotlin
package com.plcoding.readexternalmediafilesapi35

import android.net.Uri

data class MediaFile(
    val uri: Uri,
    val name: String,
    val type: MediaType
)
```

MediaType.kt | MainActivity.kt | AndroidManifest.xml

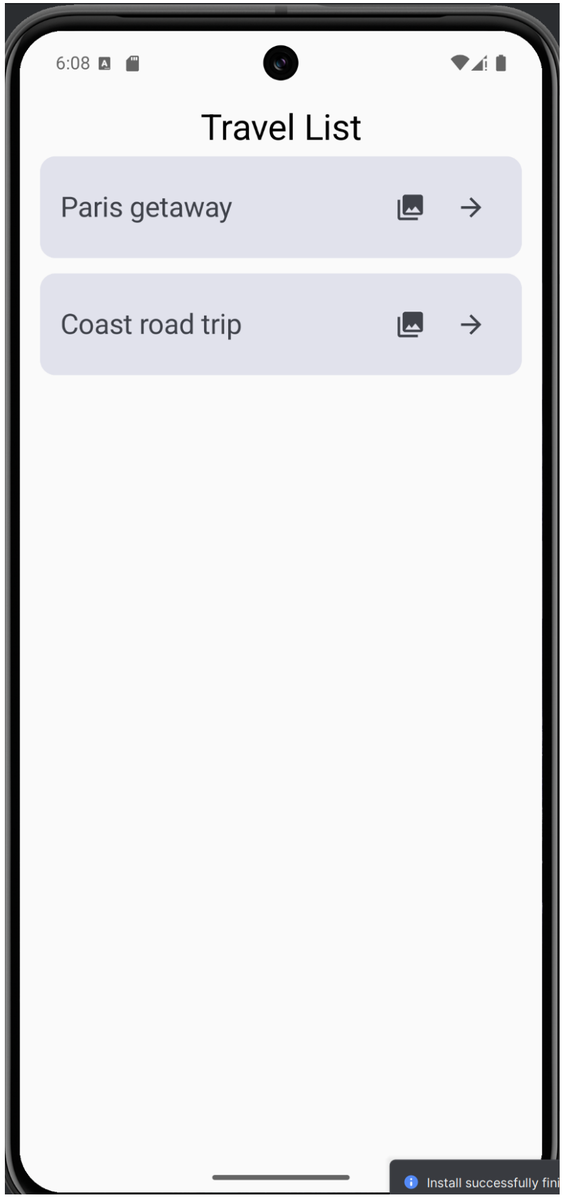
```kotlin
package com.plcoding.readexternalmediafilesapi35

enum class MediaType {
    IMAGE,
    VIDEO,
    AUDIO
}
```

```kotlin
    context.contentResolver.query(
        queryUri,
        projection,
        selection: null,
        selectionArgs: null,
        sortOrder: null
    )?.use { cursor ->
        val idColumn = cursor.getColumnIndexOrThrow(
            MediaStore.Files.FileColumns._ID
        )
        val nameColumn = cursor.getColumnIndexOrThrow(
            MediaStore.Files.FileColumns.DISPLAY_NAME
        )
        val mimeTypeColumn = cursor.getColumnIndexOrThrow(
            MediaStore.Files.FileColumns.MIME_TYPE
        )

        while(cursor.moveToNext()) {
            val id = cursor.getLong(idColumn)
            val name = cursor.getString(nameColumn)
            val mimeType = cursor.getString(mimeTypeColumn)

            if(name != null && mimeType != null) {
                val contentUri = ContentUris.withAppendedId(
                    queryUri,
                    id
                )
                val mediaType = when {
                    mimeType.startsWith( prefix: "audio/") -> MediaType.AUDIO
                    mimeType.startsWith( prefix: "video/") -> MediaType.VIDEO
                    else -> MediaType.IMAGE
                }

                mediaFiles.add(
                    MediaFile(
                        uri = contentUri,
                        name = name,
                        type = mediaType
                    )
                )
            }
        }
    }

    return mediaFiles.toList()
}
```

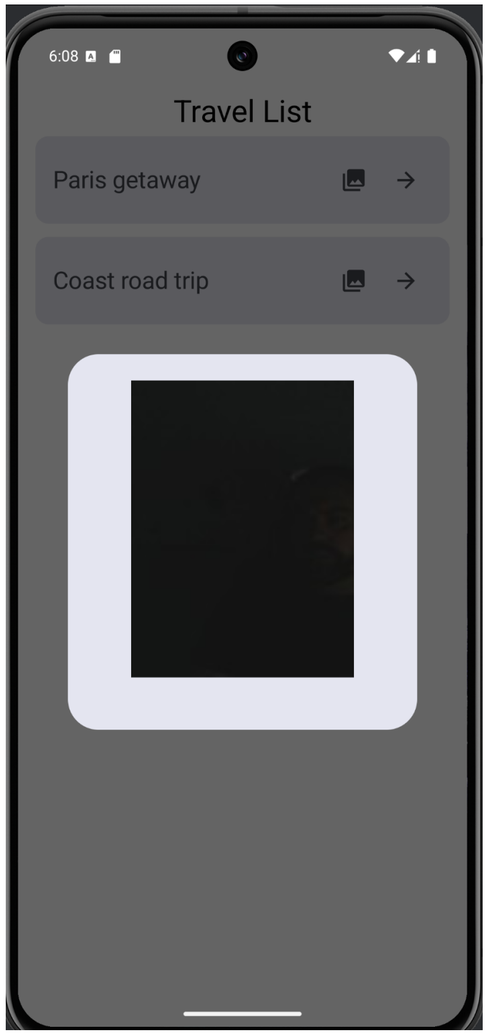
Hands On – Trip Gallery

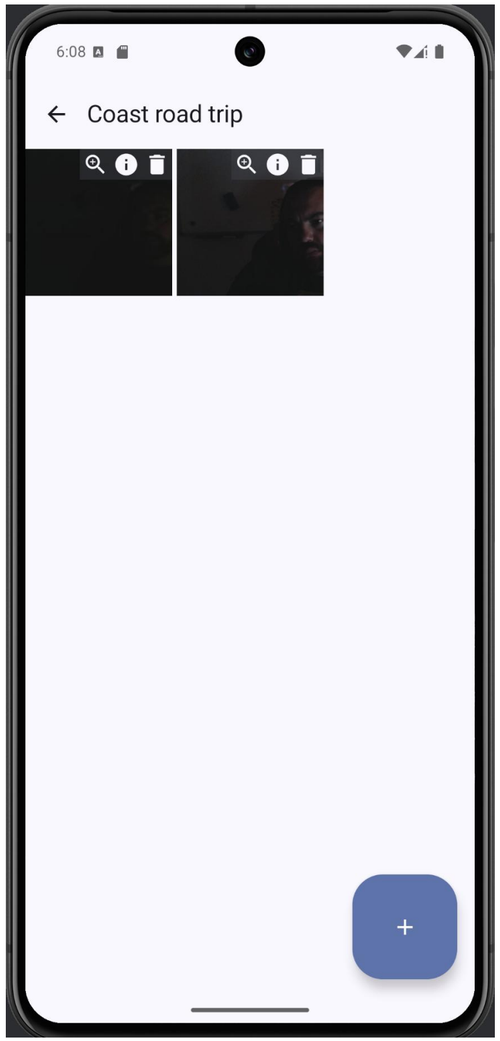
Code Review

- Download /source examples/TripGallery.zip

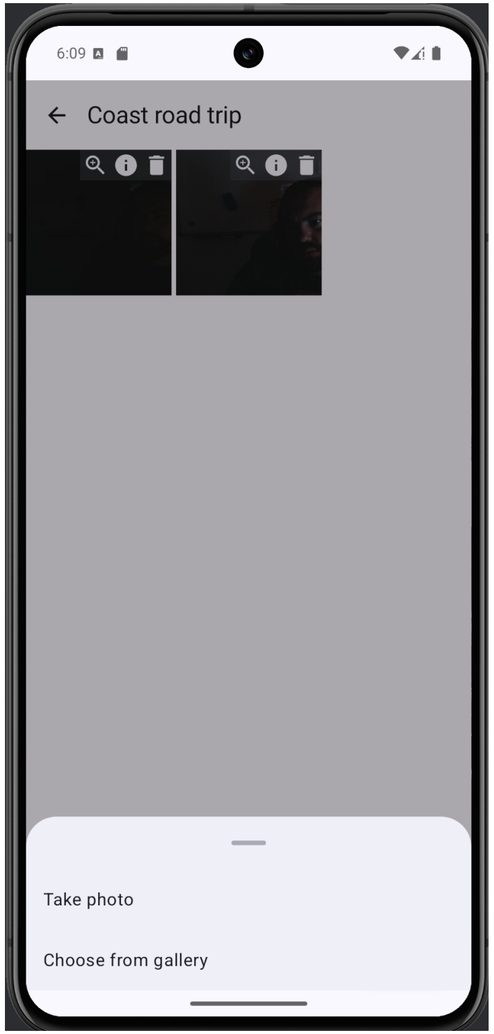


Code Review


6:08
Travel List
Paris getaway
Coast road trip


6:08
Coast road trip


6:09
Coast road trip
Image info
URI:
file:///data/user/0/com.example.tripgallery
/files/images/04f44fad-95d0-476c-977e
-4d2275ebc6b4.jpg
Size: 1 KB
Date: 2025-05-08 20:46
Close


6:09
Coast road trip
Take photo
Choose from gallery

Applications for Mobile Devices

That’s all

Full Name

2022 March 27

Universitat de Lleida

Campus Universitari Igualada - UdL